Chain-Mamba: plan from current results to a submitted paper

Chained (one-step, fed-back) prediction for model predictive control with selective state-space models

Prepared 3 November 2026 · Repository: github.com/sadi2003q/-MAMBA-STABILITY- · Companion document: `docs/chain_mamba_plan.md` · Target: draft by mid-December 2026 (about 6 weeks)

> **How to use this plan**
>
> Work through the phases in order. Each phase has a goal, the exact runs, a **pass rule written before you run**, and what to save. Do not change a pass rule after seeing a result. Write a new, dated rule instead and say so in the paper. Two results decide the paper's framing (Section 3), so they come first. Markers: ready the code already exists in the patch; needs code it has to be written first.

1. Where you are now

All runs: seed 0 on a 2-core processor, the repository's own data, windows and evaluation, a 50-step chained horizon, and about 3,300 parameters for every model. The error is the root-mean-square error averaged over the 50 steps (smaller is better).

Model (training)	Van der Pol	vs your Mamba	Four-tank (m)	vs your Mamba
Full Mamba, old training (your current setup)	0.465	—	0.0220	—
LSTM, old training	0.267	−43%	not run	
Full Mamba + new recipe	0.176	−62%	not run	
MLP + recipe (no memory)	0.102	−78%	0.0044	−80%
LSTM + recipe	0.098	−79%	0.0034	−85%
Chain-Mamba + recipe (proposed)	**0.066**	**−86%**	**0.0024**	**−89%**
Chain-Mamba, no memory margin	0.054	−88%	0.0024	−89%
Chain-Mamba, Mamba memory layer removed	0.048	−90%	0.0023	−90%

Seed 1 (Van der Pol): Chain-Mamba 0.057, no margin 0.056, LSTM + recipe 0.120. The same arm differs by about 15% between seeds.

Three findings so far

1. **The training recipe is the biggest single factor.** It means training on chained predictions with a horizon that grows from 5 to 50 steps, keeping a one-step loss alongside, and running enough updates. It lifts every network, including the LSTM.
2. **The architecture adds a further 30–60% on top of the recipe.** The pieces are the state anchor (a linear path from the current state to the next increment), output scaling by the size of true increments, and gated feed-forward (SwiGLU) layers. It beats the parameter-matched LSTM on both systems.
3. **Neither Mamba's memory nor its stability guarantee is what helps** on these two fully measured systems. Removing the margin, or the whole memory layer, did not make it worse.

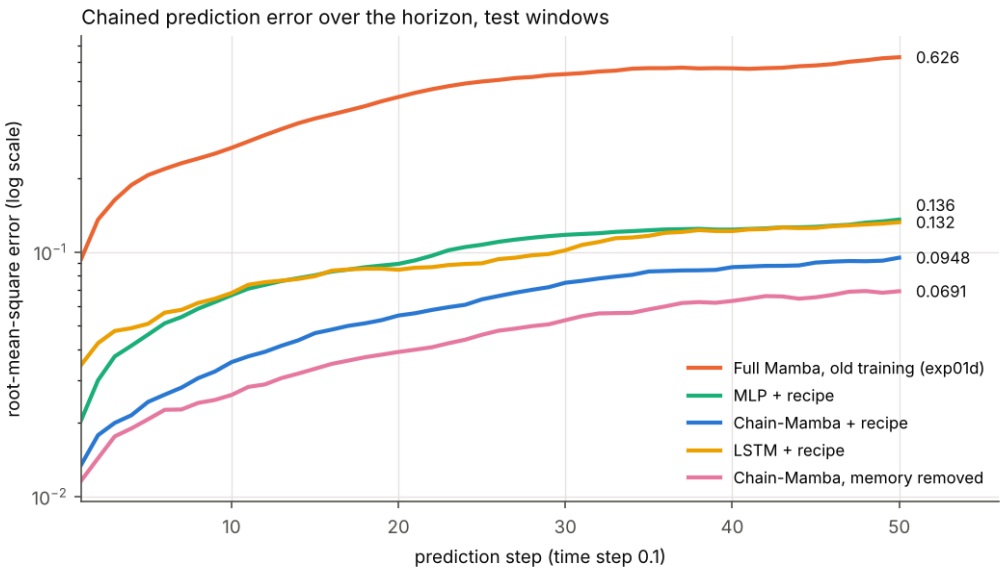


Figure A — Van der Pol, chained error at each of the 50 steps (log scale). Seed 0.

2. What the paper can claim, and what it cannot yet

Supported now (needs 8 seeds to confirm)

- Chained training with a horizon curriculum, plus a state anchor and gated feed-forward layers, cuts chained-prediction error by about 90% on two benchmark systems.
- The result beats a parameter-matched LSTM trained the same way.
- The latent stability certificate (Halloran et al.), even with an added decay margin, does not explain chained accuracy.
- Free-running limit-cycle shape and period are reproduced within a few percent (Van der Pol).

Not supported yet

- "Mamba's memory is what makes chained prediction work." The memory-free version matches it.
- "Better closed-loop control." No MPC result exists yet.
- "Better than Cevaal et al.'s direct predictor." Their design has not been trained properly as a baseline.
- Any statistical claim. Results are one or two seeds.

3. The decision that fixes the paper's framing

Run Phase 1 (8 seeds) and Phase 2 (partially measured systems) first. Phase 2 decides which paper you write.

```
Phase 2: partially measured systems (memory is needed)
 │
 ├─ Chain-Mamba ≥ 20% better than its memory-removed version on BOTH systems
 │     → PAPER A  "Chain-Mamba: selective state-space models for chained prediction in MPC"
 │        Mamba's memory earns its place when states are not all measured; on fully measured
 │        systems the anchor + recipe carry the result. Positive Mamba paper.
 │
 └─ otherwise
       → PAPER B  "What makes learned predictors work for chained prediction in MPC —
                   and why the stability guarantee of selective state-space models is not it"
          Training + anchor + gated layers work for every architecture tested; the latent
          certificate does not transfer. Strong, honest, and fits your original research question.
```

Both are publishable. Paper A is the more exciting one, and Paper B is the safer one. Most phases below are needed for both, so nothing is wasted whichever way Phase 2 goes.

4. Timeline

Week	Dates (2026)	Phases	Output by end of week
0	4–5 Nov	0 — set-up and reproduction	Seed-0 numbers reproduced on Kaggle
1	6–12 Nov	1 — 8 seeds (both systems); start 2	Statistics table; partially measured code ready
2	13–19 Nov	2 — partially measured runs; 3 — ablations	**Framing decided (A or B)**; ablation table
3	20–26 Nov	4 — Cevaal direct baseline; start 5 (MPC code)	Direct vs chained table; MPC runs on one model
4	27 Nov – 3 Dec	5 — closed-loop experiments	Stabilisation, tracking, noise, timing results
5	4–10 Dec	6 — writing (methods, results, figures)	Full draft
6	11–17 Dec	6 — revision, supervisor feedback, arXiv	Submittable manuscript

Compute: Kaggle T4 sessions are limited, so run arms in separate sessions with `--arms`; runs resume from checkpoints automatically. Processor times measured here: Chain-Mamba about 25 min per run, LSTM 10, MLP 4, full Mamba 25–70, four-tank Chain-Mamba about 55 (single core).

5. Phases

Phase 0 — Set-up and reproduction ready ½ day

Goal: make sure the numbers in Section 1 come out the same on your own machine.

```
git checkout -b chain-mamba
git apply chain_mamba_changes.patch
pytest -q                                  # expect: 36 passed
python tools/check_model.py --config configs/experiments/exp03_chain_mamba.yaml
python train.py --config configs/experiments/exp03_chain_mamba.yaml \
    --arms chain_mamba_recipe lstm_recipe --seed 0 --ask-first no
```

Pass rule: each 50-step error is within 10% of 0.066 (Chain-Mamba) and 0.098 (LSTM). Graphics-card arithmetic differs slightly from the processor's. **If it fails:** do not continue. Compare `train_log.csv` epoch by epoch with `docs/evidence/exp03_chain_mamba/runs/` to find where they part.

Phase 1 — Eight seeds on both systems ready 1–2 days of compute

Goal: turn the direction into a statistical result.

System	Arms (all chained, seeds 0–7)
Van der Pol (exp03, exp04)	`chain_mamba_recipe`, `chain_mamba_no_margin`, `chain_mamba_no_memory`, `lstm_recipe`, `mlp_recipe`, `full_mamba_recipe`
Four-tank (exp05)	`chain_mamba_recipe`, `chain_mamba_no_memory`, `lstm_recipe`, `mlp_recipe`, `full_mamba_old`

```
python train.py --config configs/experiments/exp03_chain_mamba.yaml --seeds 0 1 2 3 4 5 6 7 \
    --arms chain_mamba_recipe chain_mamba_no_margin lstm_recipe full_mamba_recipe --ask-first no
python train.py --config configs/experiments/exp04_mlp_control.yaml --seeds 0 1 2 3 4 5 6 7 --
ask-first no
python train.py --config configs/experiments/exp05_four_tank.yaml --seeds 0 1 2 3 4 5 6 7 \
    --arms chain_mamba_recipe chain_mamba_no_memory lstm_recipe mlp_recipe full_mamba_old --ask-
first no
python summarize.py --experiment outputs/exp03_chain_mamba     # and exp04, exp05
```

Pass rules (fixed now): "A is better than B" only if, across the 8 seeds, the median 50-step error is lower, the two-sided Mann–Whitney $p < 0.05$ and $|\text{Cliff's delta}| > 0.33$. Report medians with bootstrap 95% intervals. Report any run that blew up as a count, never as an error value.

- Main claim: Chain-Mamba better than LSTM + recipe, on both systems.
- Recipe claim: every recipe arm better than the same network with the old training.
- Equivalence claims ("no margin ≈ margin", "no memory ≈ Chain-Mamba"): the ratio's 95% interval must lie inside [0.90, 1.10]. Otherwise write "no difference detected".

Save: `results.csv`, `summary.md`, one table per system with medians and intervals. This becomes Table 2 of the paper.

Phase 2 — Partially measured systems: the deciding test needs code (about 1 day) 2–3 days total

Goal: find out whether Mamba's memory helps when the model cannot see the full state.

- **Van der Pol, x1 only:** the model sees measured x1 and u, and predicts x1. The velocity must be inferred from history.
- **Four-tank, levels 1 and 2 only:** these are Cevaal et al.'s usual outputs; the upper tanks are hidden.

Code to add: an `observed` list in the system config. The dataset keeps only those state columns, and the anchor and loss act on the measured outputs. The context of 10 past measurements is already there to warm up the memory. For a fair memory-free control, give it a window of the last $k = 4$ measurements and inputs as input. That is the usual way to make a memory-free model work without the full state (a NARX model); also try $k = 10$.

Arms: Chain-Mamba (margin and no margin), Chain-Mamba with memory removed plus a k-window, LSTM, all with the recipe, seeds 0–7.

Pass rule (fixed now): Chain-Mamba's median 50-step error is at least 20% below the memory-removed k-window control on **both** systems, and the Phase 1 test passes. If yes, write Paper A; if not, Paper B.

Phase 3 — Ablations: which piece does what mostly ready 1–2 days

Start from Chain-Mamba + recipe on Van der Pol and remove one piece at a time: one seed first, then 8 seeds for any change above 20%.

Arm	Config override
No anchor	`readout: {linear_skip: false}`
No output scaling	`readout: {increment_normalisation: false}`
No gated feed-forward	`model: {ffn_hidden: 0}`, width raised to keep about 3,300 parameters
Long convolution	`model: {d_conv: 10}`
Final normalisation back	`model: {final_norm: true}`
Recipe pieces	No curriculum (`rollout_curriculum` removed); no one-step loss (`teacher_aux_weight: 0`); 40 epochs
Noise injection	`state_noise_std: 0.01` (literature remedy; expected not to help)

Save: one bar chart of the 50-step error, one bar per ablation, with the full model as reference. This becomes Figure 3.

Phase 4 — Cevaal et al.'s direct predictor as a proper baseline needs code (½–1 day) 1–2 days

Goal: the comparison every reviewer will ask for. Implement their input embedding, where every token is [u(k), x0] (their Equation 22). Reset the hidden state at each call, use horizon N = 10 for Van der Pol and 20 for four-tank, and train on direct multi-step loss with the same parameter budget and epochs.

Compare with Chain-Mamba at the horizons where both apply (10, 20) and beyond the direct model's horizon (50, 100), where only chained prediction works. Also compare prediction time per MPC call.

Pass rule: none to pass. This is a reference. Report it honestly even if their predictor wins at its own horizon.

Phase 5 — Closed-loop model predictive control needs code (3–5 days) 1–2 weeks

Goal: show the prediction gain reaches the controller. Follow Cevaal et al.'s protocol so results are directly comparable. Use CasADi with IPOPT (their choice), or a PyTorch gradient-based solver with warm start as a simpler first version.

Experiment	Set-up (from Cevaal et al., Section VI)	Metrics
Stabilisation	Van der Pol, 100 initial states, $x1 \in [-2.5, 2.5]$, $x2 \in [-2, 2]$; N = 10, Q = 50, R = 0.5, P = 100	Converged count, settling time
Reference tracking	Van der Pol piecewise-constant reference; Q = 100, R = 0.5	MAE, MSE, ISE, IAE, input energy
Noise robustness	Signal-to-noise ratio 20 in data and measurements, 100 noise realisations	Mean ± standard deviation of ISE/IAE
MIMO tracking	Four-tank, inputs in [0, 4], Q = 100, R = 1, N = 20	MAE per level, constraint violations
Timing	All of the above; horizons 5 … 50	Mean solve time vs sampling time

Controllers: Chain-Mamba (chained), the LSTM + recipe (chained), Cevaal-style direct Mamba (Phase 4), and nonlinear MPC with the true model as the best achievable reference. Warm-start the memory from the last 10 measurements.

Pass rule (fixed now): Chain-Mamba has lower tracking error (median over runs) than both learned baselines on both systems, with solve time below the sampling time. If it fails, report prediction results plus the closed-loop result as it is. The paper is then a prediction paper with an honest closed-loop section.

Phase 6 — Writing 2 weeks

Write in this order: results (the tables are already made), then method, experiments, introduction, related work and abstract last. Use the outline in Section 6. Send the results section to your supervisor early in week 5.

6. Paper outline

Section	Content	Source in the repository
Abstract	Problem (chained prediction for MPC), what was wrong, the recipe and architecture, about 90% error reduction on two systems, LSTM comparison, closed-loop result, and the certificate finding	—
1. Introduction	MPC needs multi-step predictions. Direct predictors (Cevaal et al.) are fixed to their horizon, while chained ones are flexible but accumulate error. The question: what makes chained prediction work, and does Mamba's stability guarantee help? List the contributions.	README §1–4
2. Related work	Mamba and MPC (Cevaal; MamKO; bilinear Mamba-Koopman); stability of selective state-space models (Halloran; Cao et al.); simulation-error training (SUBNET; pushforward; scheduled sampling); Mamba-X and Mamba-3	`docs/chain_mamba_plan.md` §4
3. Problem set-up	Chained vs direct prediction, the error recursion, the latent certificate	README §3; `analysis/a01`
4. Diagnosis	Why the standard Mamba fails at chained prediction: training budget and curriculum, no state–input products, slow memory (decay 0.996–0.999), normalised read-out	plan §3; `tools/diagnose_memory_margin.py`
5. Method	Chain-Mamba layer equations, anchor, increment normalisation, decay margin (with the a05 bound), training recipe	plan §5–6; `analysis/a05`
6. Prediction experiments	Two systems (plus the partially measured variants), 8 seeds, Table 2, ablations (Figure 3), limit cycle (Figure 4)	Phases 1–3
7. Closed-loop experiments	Stabilisation, tracking, noise, four-tank, timing	Phase 5
8. Discussion	Paper A: when memory matters. Paper B: why the certificate does not transfer and what does. Limits: two or three systems, simulation only.	results log
9. Conclusion	Three sentences and future work (hardware, Mamba-3 rotation, stability proof for the closed loop)	—

Figures and tables to produce

#	Item	Status
Fig. 1	Chained vs direct prediction diagram, and the Chain-Mamba block diagram	to draw
Fig. 2	Error vs prediction step, both systems, median and band over 8 seeds	single-seed version exists (`tools/compare_runs.py`)
Fig. 3	Ablation bar chart	Phase 3
Fig. 4	Free-running limit cycle, Van der Pol	exists (`free_running.png`)
Fig. 5	Closed-loop tracking traces (Van der Pol, four-tank)	Phase 5
Fig. 6	Solve time vs horizon	Phase 5
Table 1	Models, parameter counts, training settings	configs
Table 2	Prediction errors (1, 10, 50 steps), medians and intervals, significance	Phase 1–2
Table 3	Closed-loop metrics	Phase 5
Table 4	Decay-factor diagnosis (memory gain) for each model	exists

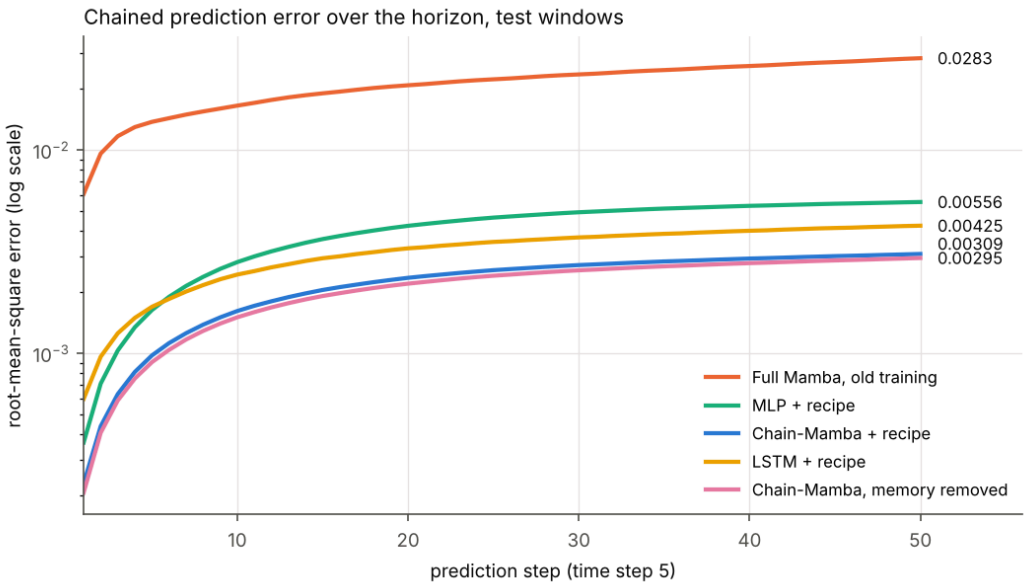


Figure B — Four-tank, chained error at each of the 50 steps (log scale). Seed 0.

7. Where to submit

Venue	Why it fits	Notes
IEEE Open Journal of Control Systems	Where de Jong, Shukla and Lazar published DeepONet-MPC, which Cevaal et al. build on; the same community and reviewers	Journal; full paper with closed loop
IEEE Control Systems Letters (L-CSS)	Short (6 pages), fast; strong for Paper B	Check current deadlines and conference options
Learning for Dynamics and Control (L4DC)	Learning-for-control audience; values careful empirical work	Conference; check the current call
arXiv (math.OC / eess.SY)	Put it out when the draft is ready, as Cevaal et al. did	Do this at the end of week 6

Check each venue's current call, page limit and dates before choosing; they change every year.

8. Risks and fallbacks

Risk	Likelihood	Fallback
8 seeds shrink the Chain-Mamba vs LSTM gap below significance	Medium	Report the recipe result (very large, certain to hold) as the main claim, and the architecture result as a tendency
Memory does not help even with partial measurements	Medium–high	Paper B. The memory-removed control becomes a finding, not a problem
MPC code takes longer than planned	Medium	Start with a PyTorch gradient solver on Van der Pol only, and add four-tank and CasADi later
Cevaal's direct predictor wins at its own horizon	Medium	That is fine. Chained prediction's selling point is any horizon and robustness beyond it; show both
Kaggle time runs out mid-grid	High	Runs resume from checkpoints; split by `--arms` and seeds
Reviewer: "this is just an MLP with a skip connection"	High (Paper B)	Ablations (Phase 3) and the plain-MLP control (0.102 vs 0.048) show the gated layers and the recipe matter. Frame the contribution as the finding, not as a new layer

9. Rules to keep the paper honest

- Write every pass rule into the config before running. After a surprise, add a new dated rule; never edit the old one.
- Always report that the LSTM improves greatly with the same recipe. The fair comparison is recipe against recipe.
- Report runs that blew up as counts; report all seeds, including bad ones.
- State the arms added after seeing results (the memory-removed control) as such.
- Keep the four-tank set-up differences from Cevaal et al. in the paper: 40,000 vs 80,000 samples, and our hold times and initial levels.
- Results from a processor and a graphics card are not mixed in one table.

10. Checklist

Experiments

- ☐ Phase 0: reproduction passes
- ☐ Phase 1: 8 seeds, Van der Pol
- ☐ Phase 1: 8 seeds, four-tank
- ☐ Phase 2 code: observed outputs, *k*-window control
- ☐ Phase 2 runs; framing decided (A / B)
- ☐ Phase 3: ablations
- ☐ Phase 4: Cevaal direct baseline
- ☐ Phase 5: MPC code
- ☐ Phase 5: stabilisation, tracking, noise, four-tank, timing

Writing

- ☐ Tables 1–4 final
- ☐ Figures 1–6 final
- ☐ Results and method drafted
- ☐ Introduction and related work
- ☐ Supervisor feedback round
- ☐ Code and checkpoints released (repository tag)
- ☐ Abstract, title, keywords
- ☐ Venue formatting
- ☐ arXiv posted

Key references

- Cevaal, de Jong, Lazar — Mamba Sequence Modeling meets Model Predictive Control (2026), arXiv:2604.13857
- Halloran, Gulati, Roysdon — Mamba State-Space Models Are Lyapunov-Stable Learners (TMLR 2025), arXiv:2406.00209
- Bilinear Input Modulation for Mamba (Mamba-X) (2026), arXiv:2604.17221 · Mamba-3 (2026), arXiv:2603.15569
- Beintema, Schoukens, Tóth — deep subspace encoders and truncated simulation loss (SUBNET), ICLR 2023
- Brandstetter, Worrall, Welling — Message Passing Neural PDE Solvers (pushforward trick) (2022), arXiv:2202.03376
- Bonassi et al. — Structured state-space models are deep Wiener models, arXiv:2312.06211
- Li, Han, Yin — MamKO: Mamba-based Koopman operator for modeling and predictive control (ICLR 2025)
- Rollout error in latent reduced-order models is accumulated phase drift (2026), arXiv:2608.07189